المدرسة الوطنية للعلوم التطبيقية بتطوان
ⵜⴰⵙⴷⴰⵡⵉⵜ ⵏ ⵜⵎⵓⵙⵙⵏⵉⵏ ⵏ ⵜⵎⵓⵏⴰⴷ
Ecole Nationale des Sciences Appliquées de Tétouan

UNIVERSITÉ ABDELMALEK ESSAÂDI
ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES DE TÉTOUAN

DÉVELOPPEMENT MOBILE

Guide Installation
Application Mobile : MyHanut
Application de Commande Anticipée auprès des Épiciers Locaux

FILIÈRE : GÉNIE INFORMATIQUE

2ème année Cycle Ingénieur

INFORMATIONS DU PROJET	
ÉQUIPE	**ENCADREMENT**
BOUCHENNOU Ferdaouss **EL ALLOUCHE Zakariyae** **TAFRAOUTI Sanae** **ZOHRI Hafsa**	**Oussama El Hajjamy** *Professeur Développement Mobile* **Livrable :** Guide Installation

PROJET ACADÉMIQUE - ANNÉE UNIVERSITAIRE 2025-2026

Table des matières

1 Guide d'installation et de configuration **1**
1.1 Introduction 1
1.2 Outils à installer 1
1.3 Cloner le dépôt 1
1.4 Base MySQL et données de test 2
1.5 Fichiers d'environnement 2
1.5.1 Exemple de `backend/.env` 2
1.5.2 Exemple de `mobile/.env` 3
1.6 Paquets et démarrage 3
1.6.1 Backend 3
1.6.2 Mobile 4
1.7 Scénario d'évaluation minimal 4
1.8 Référence des variables 4
1.9 Dépannage 5

Guide d'installation et de configuration

1.1 Introduction

Ce document décrit comment **cloner le dépôt**, reproduire la configuration des fichiers `.env`, importer la base MySQL, puis **lancer l'API Node.js** et l'**application Flutter** MyHanut.

Contenu du dépôt :

- `backend/` — API Node.js (Express, Sequelize, MySQL) ;
- `mobile/` — application Flutter ;
- `epicier_ecommerce.sql` — schéma MySQL et données de test (à importer).

Les fichiers `backend/.env.example` et `mobile/.env.example` sont des **modèles génériques**. Les valeurs concrètes à utiliser après clonage sont détaillées à la section 1.5.

1.2 Outils à installer

Outil	Usage
Git	Cloner le dépôt
Node.js + npm	Dossier `backend/`
MySQL	Base de données
Flutter (SDK $\geq$ 3.11)	Dossier `mobile/` (voir `pubspec.yaml`)
JDK 17	Build Android
Android Studio (ou Xcode sur Mac)	Émulateur / SDK

Table 1.1 – Environnement de développement recommandé

Vérifications : `node -v`, `npm -v`, `mysql -version`, `flutter doctor`.

1.3 Cloner le dépôt

```
git clone https://github.com/ferdaoussBouchennou/Application-Mobile-
    ↪ Commande-Epicier.git
cd Application-Mobile-Commande-Epicier
```

1.4 Base MySQL et données de test

Le backend utilise par défaut une base nommée `epicier_ecommerce` (variable `DB_NAME` dans `backend/.env`). Adapter si nécessaire.

```
mysql -u root -p -e "CREATE DATABASE IF NOT EXISTS epicier_ecommerce
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;"
mysql -u root -p epicier_ecommerce < epicier_ecommerce.sql
```

Comptes de démo

- Administrateur : `admin@gmail.com` / `12345678`
- Épicier : `ahmed@hanut.com` / `Password123`
- Client : `ismail@gmail.com` / `Ismail@12`

1.5 Fichiers d'environnement

Fichier	Rôle
`backend/.env`	Variables lues par Node (`dotenv`)
`mobile/.env`	Lu par Flutter (`flutter_dotenv`) ; certaines clés peuvent être lues par Gradle (Android)
`serviceAccountKey.json`	`backend/src/config/firebase/` — Firebase Admin (push), hors `.env`

Table 1.2 – Emplacements des fichiers sensibles

Les fichiers `.env` ne sont en principe **pas versionnés** (`.gitignore`). Les créer localement à partir des modèles et du contenu de référence ci-dessous.

1.5.1 Exemple de `backend/.env`

```
# ================================
# CONFIGURATION SERVEUR
# ================================
PORT=3000
NODE_ENV=development
JWT_SECRET=super_secret_jwt_key_myhanut_2026_xyz

# ================================
# BASE DE DONNÉES (MySQL)
# ================================
DB_HOST=localhost
DB_PORT=3306
DB_NAME=epicier_ecommerce
DB_USER=root
DB_PASS=

# ================================
# EMAIL SMTP (verification & mot de passe oublie)
# ================================
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
```

```
SMTP_USER=boferdaous@gmail.com
SMTP_PASS=urlw jkzcobdtozsx
SMTP_FROM="MyHanut <boferdaous@gmail.com>"

# Inscription par facebook et instagram
FACEBOOK_APP_ID=26107538408906257
FACEBOOK_APP_SECRET=9bc91bb55c65fa4b7866312ef0874da2
INSTAGRAM_APP_ID=26107538408906257
INSTAGRAM_APP_SECRET=9bc91bb55c65fa4b7866312ef0874da2

# Inscription par google
GOOGLE_CLIENT_ID=601186164981-j98iutbsm8bm608vkmn2i8b7889kcffe.apps.googleusercontent.com
```

Remarques : `DB_PASS` peut rester vide si MySQL n'a pas de mot de passe. Pour Gmail, utiliser un **mot de passe d'application**. `GOOGLE_CLIENT_ID` doit être **identique** entre backend et mobile pour la connexion Google.

1.5.2 Exemple de `mobile/.env`

```
# ================================
# CONFIGURATION API
# ================================
# URL de votre backend
# Sur emulateur Android  -> http://10.0.2.2:3000
# Sur simulateur iOS     -> http://127.0.0.1:3000
# Sur vrai telephone     -> http://<IP_de_votre_PC>:3000
# En production          -> https://api.myhanut.com
API_BASE_URL=http://localhost:3000/api
#API_BASE_URL=http://10.0.2.2:3000/api

# Inscription par facebook et instagram
FACEBOOK_APP_ID=26107538408906257
FACEBOOK_APP_SECRET=9bc91bb55c65fa4b7866312ef0874da2
FACEBOOK_CLIENT_TOKEN=fda07fe763d3514b15119afb03a6b08b
INSTAGRAM_APP_ID=26107538408906257
INSTAGRAM_APP_SECRET=9bc91bb55c65fa4b7866312ef0874da2

# Inscription google
GOOGLE_CLIENT_ID=601186164981-j98iutbsm8bm608vkmn2i8b7889kcffe.apps.googleusercontent.com
```

Remarques : `API_BASE_URL` doit **toujours** se terminer par **`/api`**. Sur **émulateur Android**, `10.0.2.2` pointe vers la machine hôte. Sur **simulateur iOS**, utiliser `http://127.0.0.1:3000/api`. Sur **téléphone réel**, remplacer par `http://<IP_LAN_du_PC>:3000/api` (pare-feu ouvert sur le port 3000).

1.6 Paquets et démarrage

1.6.1 Backend

```
cd backend
npm install
npm run dev
```

Test : ouvrir `http://localhost:3000` — une réponse JSON du type « API is running » confirme le bon fonctionnement.

1.6.2 Mobile

```
cd mobile
flutter pub get
flutter run
```

Exécution Web & Authentifications Sociales : Pour que l'authentification avec Google fonctionne dans le web, lancer l'application avec la commande suivante :

```
flutter run --web-port 5000
```

Attention : L'authentification via Facebook et Instagram fonctionne **seulement sur le mobile** (pas sur le web). Ces deux options partagent la même fenêtre de connexion car Meta a lié Instagram avec Facebook.

1.7 Scénario d'évaluation minimal

1. Importer le SQL dans une base dont le nom = `DB_NAME`.
2. `JWT_SECRET` et `DB_*` corrects ⇒ l'API démarre.
3. `API_BASE_URL` adapté à l'émulateur ou au téléphone ⇒ l'application joint l'API.
4. Connexion e-mail / mot de passe avec les comptes du fichier SQL possible sans configurer Maps ni Facebook.
5. Google Sign-In : nécessite `GOOGLE_CLIENT_ID` cohérent.
6. Firebase / FCM : optionnel au premier test (l'application gère l'échec d'initialisation).

1.8 Référence des variables

Variable	Obligatoire ?	Où c'est lu
`DB_NAME`, `DB_USER`, `DB_PASS`	Oui	`src/config/db.js`
`DB_HOST`, `DB_PORT`	Non (défauts)	idem
`JWT_SECRET`	**Oui**	`src/middlewares/auth.js`, `src/controllers/authController.js`
`PORT`, `NODE_ENV`	Non	`src/app.js`
`GOOGLE_CLIENT_ID`	Si test Google	`src/controllers/authController.js`
`SMTP_*`	Non (e-mails)	`src/utils/emailService.js`

Table 1.3 – Variables backend (`backend/.env`)

Variable	Rôle
`API_BASE_URL`	`lib/core/constants/api_constants.dart`
`GOOGLE_CLIENT_ID`	`lib/providers/auth_provider.dart`
`FACEBOOK_APP_ID`, `FACEBOOK_CLIENT_TOKEN`	Build Android (`android/app/build.gradle.kts`)

Table 1.4 – Variables mobile (`mobile/.env`)

1.9 Dépannage

Symptôme	Piste
Backend quitte tout de suite	`JWT_SECRET` manquant
Erreur MySQL	`DB_NAME` / utilisateur / mot de passe ; base non importée
App n'atteint pas l'API	`API_BASE_URL` (`10.0.2.2` vs `127.0.0.1` vs IP LAN), pare-feu port 3000
Google Sign-In	`GOOGLE_CLIENT_ID` identique backend + mobile
E-mails non envoyés	`SMTP_PASS` (mot de passe d'application), `SMTP_USER`